

Matrici

Trasposta

Versione con sintassi array con Mx di partenza e destinazione

```
void invert(int m[ROW][COL], int m2[ROW][COL]) {  
    for (int i = 0; i < ROW; i++) {  
        for (int j = 0; j < COL; j++) {  
            m[i][j] = m2[j][i];  
        }  
    }  
}
```

Versione con puntatori con Mx di partenza e destinazione

```
void trasponi(int *Mx, int *MxT, int row, int col) {  
    for (int r=0; r<row; r++) {  
        for (int c=0; c<col; c++) {  
            *(MxT+r+c*row)=*(Mx+c+r*col);  
        }  
    }  
}
```

Versione con sintassi array e "copia sul posto"

```
void invert_v2(int m[ROW][COL]) {  
    int temp;  
    for (int r = 0; r < ROW; r++) {  
        for (int c = r + 1; c < COL; c++) {  
            temp = m[c][r];  
            m[c][r] = m[r][c];  
            m[r][c] = temp;  
        }  
    }  
}
```

Somma di matrici

Le matrici su cui effettuare la somma devono avere le stesse dimensioni ($N \times M$).

Con i due cicli nidificati, ciascuno sulle dimensioni di righe e colonne, si accede alla medesima posizione (i,j) e si assegna il risultato sempre nella stessa posizione

```
#include <stdio.h>
#include <stdlib.h>

#define N 4 // Numero di righe
#define M 3 // Numero di colonne

void add(int A[][M], int B[][M], int C[][M]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }
}

int main() {
    int A[N][M] = { {1, 1, 1}, {2, 2, 2}, {3, 3, 3}, {4, 4, 4} };
    int B[N][M] = { {1, 1, 1}, {2, 2, 2}, {3, 3, 3}, {4, 4, 4} };
    int C[N][M];

    add(A, B, C);

    printf("Result matrix is:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Contare tutte le righe ordinate in una matrice

```
#include <stdio.h>
#include <stdbool.h>
#define dim 3

int isOrdinato(int Matrix[dim][dim]){
    int conta = 0;
    bool ordinato;
    for(int i=0;i<dim;i++){
        ordinato = true;
        for(int j=0;j<dim - 1;j++){
            if (Matrix[i][j] > Matrix[i][j+1]){
                ordinato = false;
            }
        }
    }
}
```

```

        if (ordinato){
            conta++;
        }
    }
    return conta;
}

int main()
{
    int matrice[dim][dim] = {{1,2,3},{4,7,3},{3,9,11}};
    printf("\n%d",isOrdinato(matrice));

    return 0;
}

```

Stampa matrici

```

#define ROW (2)
#define COL (2)

void printMatrix(int Mx[ROW][COL]) {
    for (int v = 0; v < ROW; v++) {
        for (int z = 0; z < COL; z++) {
            printf("%d ", Mx[v][z]);
            fflush(stdout);
        }
        printf("\n");
    }
}

```

oppure versione con puntatori

```

void printMatrixPt(int *Mx, int row, int col) {
    for (int v = 0; v < row; v++) {
        for (int z = 0; z < col; z++) {
            printf("%d ", *(Mx+z+v*col));
            fflush(stdout);
        }
        printf("\n");
    }
}

```

Prodotto Scalare

```
void prodScal(int m[ROW][COL], int prod) {
    for (int i = 0; i < ROW; i++) {
        for (int j = 0; j < COL; j++) {
            m[i][j] = m[i][j] * prod;
        }
    }
}
```

Prodotto Matrice x Vettore

Si hanno una matrice e un vettore:

- Mx di dimensione $m \times n$
- V di dimensione n
- La matrice deve avere il numero di colonne uguale al numero di elementi del vettore, n in questo caso.

Il prodotto è un vettore di dimensione m , in cui l'elemento **i-mo** si calcola in base alla **riga i-ma** di A e facendo la **somma dei prodotti elemento per elemento**:

$$c_i = \sum_{k=1}^n a_{ik} b_k$$

ovvero:

Per ogni elemento del vettore risultato:

1. data una **riga** della prima matrice
2. **moltiplicare** gli elementi corrispondenti della riga della Mx e quelli del vettore,
3. **sommare** questi prodotti: il risultato è l'elemento del nuovo vettore.

Esempio

$$A (2 \times 3): \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$B (3 \times 1): \begin{bmatrix} 7 \\ 9 \\ 11 \end{bmatrix}$$

$$\text{Prodotto } C = A \times B = A \times B (2 \times 1):$$

$$c_{11} = 1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 = 58$$

$$c_{21} = 4 \cdot 7 + 5 \cdot 9 + 6 \cdot 11 = 139$$

$$C = \begin{bmatrix} 58 \\ 139 \end{bmatrix}$$

```
void MxPerVet(int Mx[ROWA][COLA], int vet[ROWB], int ris[ROWA]) {
    for (int r = 0; r < ROWA; r++) {
        ris[r] = 0;
        for (int c = 0; c < COLA; c++) {
            ris[r] = ris[r] + (Mx[r][c] * vet[c]);
        }
    }
}
```

```
}  
}
```

Prodotto di due matrici

Si hanno due matrici:

- A di dimensione $m \times n$
- B di dimensione $n \times p$
- La prima matrice deve avere il numero di colonne uguale al numero di righe della seconda, n in questo caso

il **prodotto delle matrici** $C = A \times B$ è una matrice di dimensione $m \times p$, in cui l'**elemento** c_{ij} (riga i, colonna j) si calcola prendendo:

- la **riga i** di A
 - la **colonna j** di B
- e facendo la **somma dei prodotti elemento per elemento**:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

ovvero:

Per ogni elemento della matrice risultato:

1. data una **riga** della prima matrice,
2. data una **colonna** della seconda matrice,
3. **moltiplicare** gli elementi corrispondenti della riga e della colonna,
4. **sommare** questi prodotti: il risultato è l'elemento della nuova matrice.

Esempio

$$A (2 \times 3): \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$B (3 \times 2): \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}$$

Prodotto $C = A \times B = A \times B (2 \times 2)$:

$$c_{11} = 1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 = 58$$

$$c_{12} = 1 \cdot 8 + 2 \cdot 10 + 3 \cdot 12 = 64$$

$$c_{21} = 4 \cdot 7 + 5 \cdot 9 + 6 \cdot 11 = 139$$

$$c_{22} = 4 \cdot 8 + 5 \cdot 10 + 6 \cdot 12 = 154$$

$$C = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$$

Versione diretta

```
#include <stdio.h>
#include <stdlib.h>

#define R1 2 // number of rows in Matrix-1
#define C1 2 // number of columns in Matrix-1
#define R2 2 // number of rows in Matrix-2
#define C2 2 // number of columns in Matrix-2

void mulMat(int m1[R1][C1], int m2[R2][C2], int rslt[R1][C2]) {
    if (C1 != R2) {
        printf("Invalid Input");
        return;
    }

    for (int i = 0; i < R1; i++) {
        for (int j = 0; j < C2; j++) {
            rslt[i][j] = 0;
            for (int k = 0; k < C1; k++) {
                rslt[i][j] += m1[i][k] * m2[k][j];
            }
        }
    }
}
```

Versione con riutilizzo di Mx x Vet e Trasposta

Utilizzando la funzione della moltiplicazione matrice x vettore. Dovendo iterare sulle colonne della matrice B che si è ipotizzando essere l'indice più interno, ciò non è possibile effettuare: $MxB[][x]$ non rappresenta una colonna. Usando la funzione trasposta inverte colonne con righe quindi $MxB[x]$ rappresenta una colonna trasposta su riga

```
void MxPerMxBis(int MxA[ROWA][COLA], int MxB[ROWB][COLB], int MxRis[ROWA][COLB]) {
    int MxBT[COLB][ROWB];
    int MxRisT[COLB][ROWA];
    trasponi(MxB, MxBT, ROWB, COLB);
    for (int cB = 0; cB < COLB; cB++) {
        MxPerVet(MxA, MxBT[cB], MxRisT[cB]);
    }
    trasponi(MxRisT, MxRis, COLB, ROWA);
}
```

Triangolarizzazione di matrici

```
void triang(float mx[LEN][LEN]) {
```

```

for (int k = 0; k < LEN - 1; k++)
    for (int j = k + 1; j < LEN; j++) {
        float coef = -mx[j][k] / mx[k][k];
        for (int i = 0; i < LEN; i++) {
            mx[j][i] += coef * mx[k][i];
        }
    }
}

```

Diagonalizzazione di matrici

```

// Diagonalizzazione di una matrice
#include<stdio.h>
#define dim 4
int stampa(float matrice[dim][dim]){
    for(int i=0; i<dim; i++){
        for(int j=0; j<dim; j++){
            printf("%.0f ",matrice[i][j]);
        }
        printf("\n");
    }
}
int sommarighe(float mx[dim][dim], int tomolt, int toadd, int col){
    float k=-mx[toadd][col]/mx[tomolt][col];
    for(int i=0; i<dim; i++){
        mx[toadd][i]+=k*mx[tomolt][i];
    }
}
int diag() {
    float triangolo[dim][dim]={3,8,9,2},{9,16,18,4},{18,32,27,15},{12,64,81,44}};
    //sommarighe(triangolo,0,1,0);
    for(int k=0; k<dim-1;k++){
        for(int i=k+1; i<dim; i++){
            sommarighe(triangolo,k,i,k);
        }
    }
    for(int k=dim-1; k>0; k--){
        for(int i=k-1; i>=0; i--){
            sommarighe(triangolo,k,i,k);
        }
    }
    stampa(triangolo);
}

```

Somme delle diagonali principale e secondaria di una matrice

```

#include <stdio.h>
#define M 3

void sommaDiagonali(int matrice[M][M]) {
    int sdp = 0;
    int sds = 0;
    for (int i = 0; i <= M-1; i++) {
        sdp += matrice[i][i];
        sds += matrice[i][M-1-i];
    }

    printf("la somma della diagonale principale è : %d\n", sdp);
    printf("la somma della diagonale secondaria è : %d", sds);
}

int main()
{
    int matrice[M][M]={
        {2,2,3},
        {4,5,6},
        {7,8,9}
    };
    sommaDiagonali(matrice);
    return 0;
}

```

Contare tutte le righe ordinate in una matrice

```

#include <stdio.h>
#include <stdbool.h>
#define dim 3

int isOrdinato(int Matrix[dim][dim]){
    int conta = 0;
    bool ordinato;
    for(int i=0;i<dim;i++){
        ordinato = true;
        for(int j=0;j<dim - 1;j++){
            if (Matrix[i][j] > Matrix[i][j+1]){
                ordinato = false;
            }
        }
        if (ordinato){
            conta++;
        }
    }
    return conta;
}

```

```
int main()
{
    int matrice[dim][dim] = {{1,2,3},{4,7,3},{3,9,11}};
    printf("\n%d",isOrdinato(matrice));

    return 0;
}
```
